



ITESO, Universidad Jesuita de Guadalajara

Documento Técnico:
MQTT con reconocimiento de comandos por voz
IoT para Linux Embebido

Rodrigo Ramos Romero IE715082
Daniel
17/11/2025

Zamora Davalos Ricardo

índice

Resumen.....	3
Introducción	3
Marco teórico.....	3
Diseño del proyecto	4
Implementación.....	4
Resultados y validación	6
Conclusiones.....	10
Referencias.....	10

Resumen

Este proyecto es una aplicación basada en la PICO IMX8MM de NXP. Busca simular una computadora inteligente tipo “Alexa” con el reconocimiento de voz para controlar el acceso de la puerta (Comandos “Open y Close”) y el control de la luz (Comandos “On y Off”).

Los comandos se escuchan con el micrófono, se descomponen en sus características y se analizan para su reconocimiento. El comando que sea leído produce un mensaje en tópicos de MQTT que se pueden ver en el celular, y que se mueven a través del bróker o servidor de Mosquitto que se crea en la computadora.

Introducción

El objetivo general:

- Crear un asistente inteligente a través de comandos de voz.

Objetivos específicos

- Utilizar los conceptos ya vistos en clase para crear un proyecto mas cercano a lo que se hace en la industria (Embebidos y redes).
- Entender y utilizar el Hardware disponible de la tarjeta, el cual es diferente a lo que siempre se ha hecho de manipular señales y puertos GPIO

El proyecto es relevante, ya que es algo muy usado en la realidad. Cada vez son las populares los asistentes de este tipo, y reconocer comandos de voz y mensajes de MQTT es lo más básico de lo que se espera de estos dispositivos.

Marco teórico

MQTT: MQTT (Message Queuing Telemetry Transport) es un protocolo de comunicación ligero, basado en el modelo publicador–suscriptor, diseñado para dispositivos embebidos, IoT, sensores y redes de baja capacidad. Funciona sobre TCP/IP y se caracteriza por su bajo consumo de ancho de banda, baja sobrecarga y alta eficiencia energética.

Mosquitto (Servidor o broker de MQTT):

Eclipse Mosquitto es un servidor (broker) MQTT de código abierto que implementa completamente los estándares MQTT 3.1, 3.1.1 y 5.0. Su función es recibir mensajes de los publicadores y distribuirlos a los suscriptores correspondientes según los tópicos. Es uno de los brokers MQTT más usados en desarrollos IoT por su simplicidad y estabilidad.

Tópico de MQTT

Un tópico (topic) MQTT es una cadena de texto jerárquica que se usa para clasificar y direccionar mensajes dentro del broker. Los suscriptores eligen a qué tópicos escuchar y los publicadores envían mensajes a esos tópicos.

Suscriptor de MQTT (Subscriber)

Un suscriptor MQTT es un dispositivo, aplicación o proceso que se registra a uno o varios tópicos en el broker para recibir mensajes de ellos. El broker le enviará cualquier mensaje publicado en los tópicos a los que se haya suscrito.

Publicador de MQTT (Publisher)

Un publicador MQTT es un dispositivo o programa que envía (publica) mensajes a un tópico específico dentro del broker. Los mensajes serán entregados por el broker a todos los suscriptores de ese tópico.

Redes Neuronales

Una Red Neuronal Artificial (RNA) es un modelo computacional inspirado en la estructura biológica del cerebro, compuesto por nodos (neuronas) interconectados organizados en capas. Sirve para aprender patrones, clasificar datos, realizar predicciones, y es la base del Deep Learning. Tipos comunes:

- Perceptrón multicapa (MLP)
- Redes convolucionales (CNN)
- Redes recurrentes (RNN)

Funcionan entrenando sus parámetros (pesos y sesgos) para minimizar un error mediante un algoritmo como backpropagation.

Diseño del proyecto

El Hardware utilizado es únicamente la tarjeta de sonido de la Pico, el “Voice Hat”. Se utilizan los botones, el micrófono y las bocinas.

El software se puede dividir en varias partes:

- MQTT, para suscribirse y publicar en los tópicos correspondientes.
- Botones, para grabar, reproducir y reconocer el audio guardado.
- Reconocimiento de voz, donde se descompone el comando grabado en sus características y se comparan valores, para mandar el publish correcto de MQTT.

Implementación

Al final se cambio de usar reconocimiento de voz a reconocimiento de sonidos. Fue un cambio necesario, ya que los comandos de voz eran demasiado parecidos entre si. Esto complicaba demasiado el reconocimiento de voz para el poder de procesamiento de la Pico. El modelo funcionaba perfectamente en la computadora, como se ve en la siguiente imagen:

```
Windows PowerShell
Example test:
Detected: close
(venv) PS C:\Keyword_spotting_model> python train_keyword_model_sklearn.py
Loading WAV files...
Training KNN classifier...

Accuracy: 81.25%
Model saved as keyword_model.joblib

Example test:
Detected: off
(venv) PS C:\Keyword_spotting_model> python train_keyword_model_sklearn.py
Loading WAV files...
Training KNN classifier...

Accuracy: 81.25%
Model saved as keyword_model.joblib

Example test:
Detected: on
(venv) PS C:\Keyword_spotting_model> python train_keyword_model_sklearn.py
Loading WAV files...
Training KNN classifier...

Accuracy: 81.25%
Model saved as keyword_model.joblib

Example test:
Detected: open
(venv) PS C:\Keyword_spotting_model> |
```

Ilustración 1: Modelo de reconocimiento de voz en Python

Sin embargo, había muchos errores corriendo esta versión en la Pico. Por ello se cambió a usar sonidos.

- Open: revolver cartas
- Close: golpecitos a la mesa.
- On: chasquidos
- Off: Chiflar

Este cambio llevo a utilizar un modelo nuevo. Este genera líneas así:

```
80 6 3
0.04286103 0.00026042 0.15111899 0.16029170 -0.07219527 3.00000000 0
0.04843220 0.00080210 0.15454674 0.19464797 -0.07180480 3.00000000 0
0.03848838 0.00036459 0.13978143 0.16993115 -0.07402853 3.00000000 0
0.03414846 0.00144795 0.13830532 0.14626120 -0.06948696 3.00000000 0
...
```

Estas líneas contienen las siguientes características:

- Energy: que tan fuerte es el Sonido durante su duración.
- zero crossing rate: Cuantas veces cruza por el cero la señal, o cuantas veces cambia de signo. Mientas más alto este valor, más ruidoso es el sonido. Un valor bajo implica un sonido “Limpio” o más claro.

- mean absolute amplitud: Este promedio de amplitud nos da valores menos sensibles a picos, complementa la energía siendo menos sensible a valores rápidos pero fuertes.
- standard deviation: Que tanto se desvía del promedio el sonido. Alto en sonidos de corta duración.
- slope of envelope: La pendiente de la amplitud, es muy importante porque es posiblemente el que más varía con cada tipo de sonido. Cambia por su duración, fuerza, ruido, etc.
- Duration: No cuánto dura la grabación (3s para todos) si no, cuánto dura el sonido dentro de la grabación.

Y al final un índice para saber a qué comando pertenece. El código embebido descompone el comando en estos valores, y checa fila por fila hasta encontrar lo que se parece más, utilizando el índice al final para ver cual es Publish que tiene que mandar.

Resultados y validación

Primero, tenemos el scatterplot de los sonidos, que nos permite ver que tanto se diferencian unos de los otros.

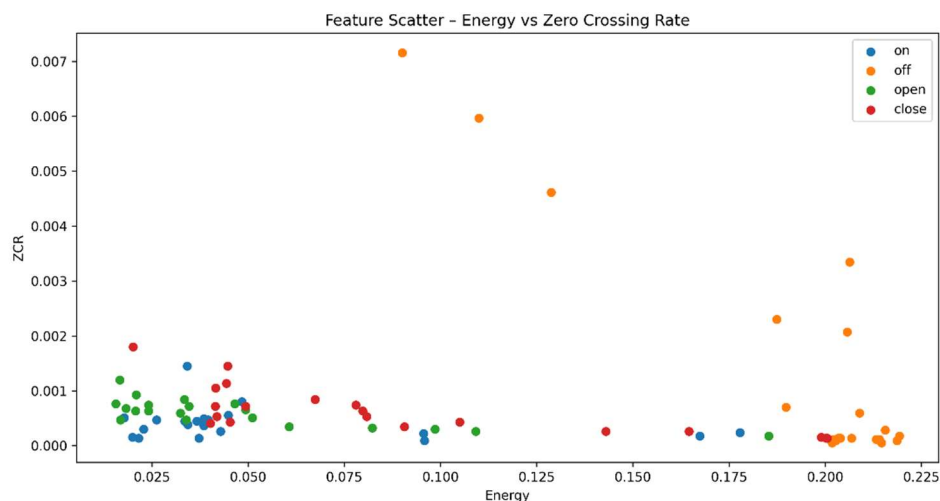
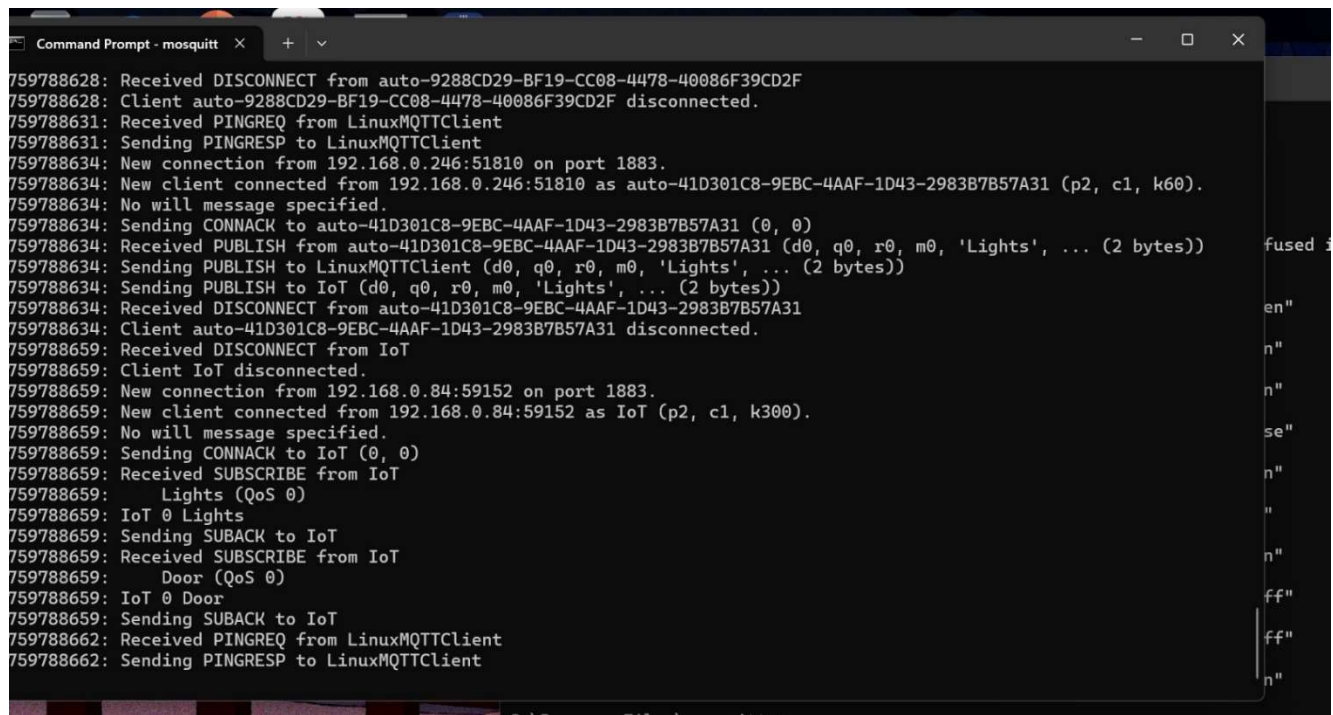


Ilustración 2: Scatterplot de los sonidos

Las pruebas se realizaron primero probando el bróker creado de MQTT con Mosquitto, usando pings, subscribes y publishes desde consola en esta primera foto vemos el servidor funcionando perfectamente y como se necesita para la aplicación.



```
759788628: Received DISCONNECT from auto-9288CD29-BF19-CC08-4478-40086F39CD2F
759788628: Client auto-9288CD29-BF19-CC08-4478-40086F39CD2F disconnected.
759788631: Received PINGREQ from LinuxMQTTClient
759788631: Sending PINGRESP to LinuxMQTTClient
759788634: New connection from 192.168.0.246:51810 on port 1883.
759788634: New client connected from 192.168.0.246:51810 as auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (p2, c1, k60).
759788634: No will message specified.
759788634: Sending CONNACK to auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (0, 0)
759788634: Received PUBLISH from auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Sending PUBLISH to LinuxMQTTClient (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Sending PUBLISH to IoT (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Received DISCONNECT from auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31
759788634: Client auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 disconnected.
759788659: Received DISCONNECT from IoT
759788659: Client IoT disconnected.
759788659: New connection from 192.168.0.84:59152 on port 1883.
759788659: New client connected from 192.168.0.84:59152 as IoT (p2, c1, k300).
759788659: No will message specified.
759788659: Sending CONNACK to IoT (0, 0)
759788659: Received SUBSCRIBE from IoT
759788659: Lights (QoS 0)
759788659: IoT 0 Lights
759788659: Sending SUBACK to IoT
759788659: Received SUBSCRIBE from IoT
759788659: Door (QoS 0)
759788659: IoT 0 Door
759788659: Sending SUBACK to IoT
759788662: Received PINGREQ from LinuxMQTTClient
759788662: Sending PINGRESP to LinuxMQTTClient
```

Ilustración 3: Broker MQTT

Teniendo un bróker de MQTT funcional, el siguiente paso es preparar la interfaz. La meta del proyecto es lograr una interfaz completa con el celular, imprimiendo información de estados en este, y permitiendo actualizar estados y salidas del sistema total desde el celular. Se utilizó la aplicación “MQTT Panel” y esta es la interfaz creada:

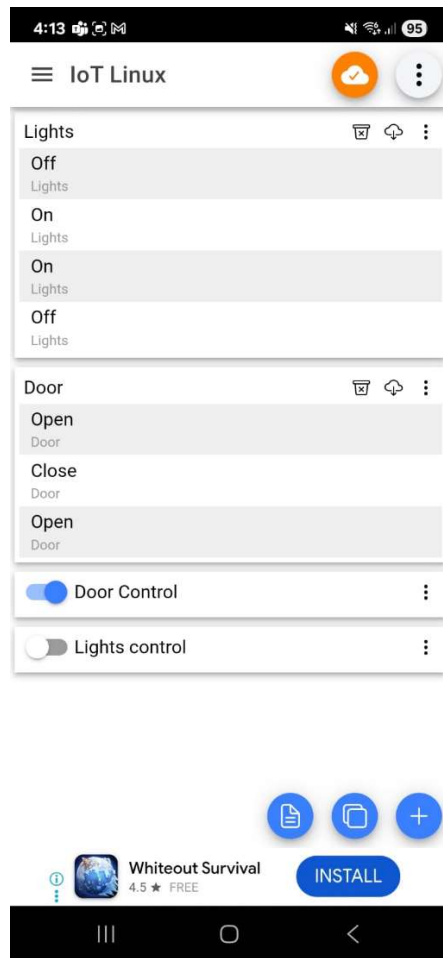


Ilustración 4: Aplicacion de Celular

Aquí podemos ver ya la aplicación funcionando. Imprimimos los últimos 3 estados o cambios de los tópicos “Door” y “Lights”. Los switches nos permiten actualizar manualmente el estado.

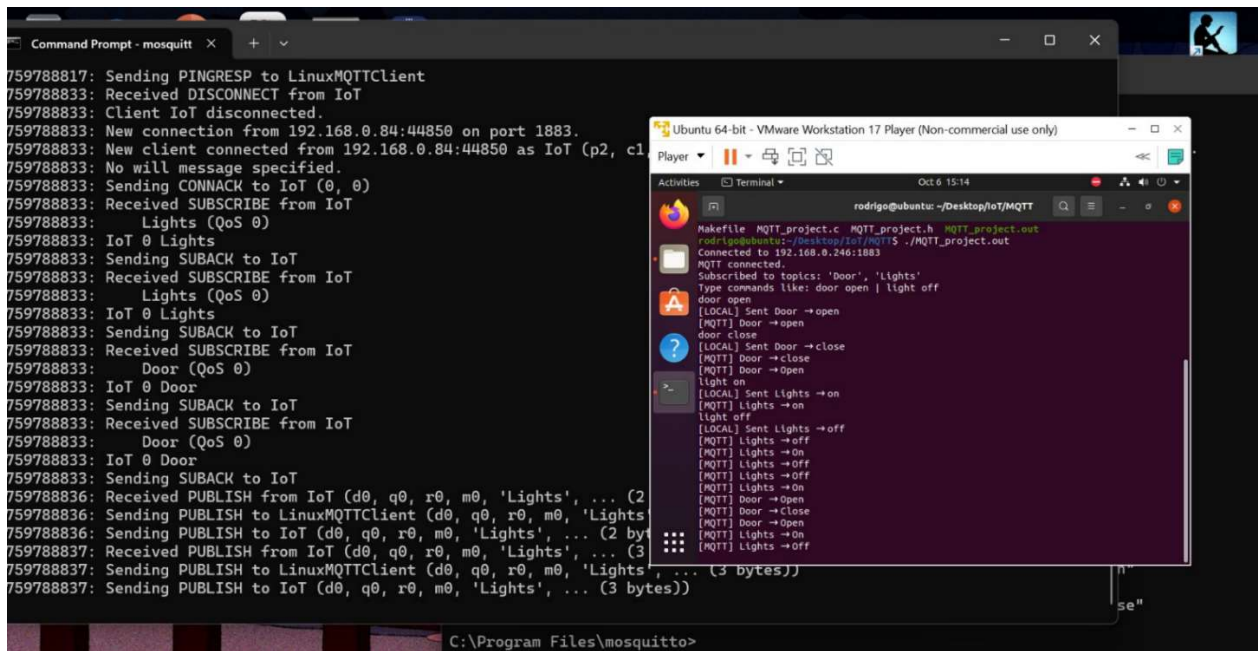


Ilustración 5: Aplicacion inicial con Broker

La aplicación funciona como fue esperado y definido en las metas. Se puede actualizar en la misma aplicación el estado de los dos tópicos o funcionalidades.

En esta foto podemos ver que el reconocimiento de voz en su versión original funcionaba muy bien en PC. Podemos ver que se reconocen bien los 4 comandos a pesar de su precisión de 81.25%

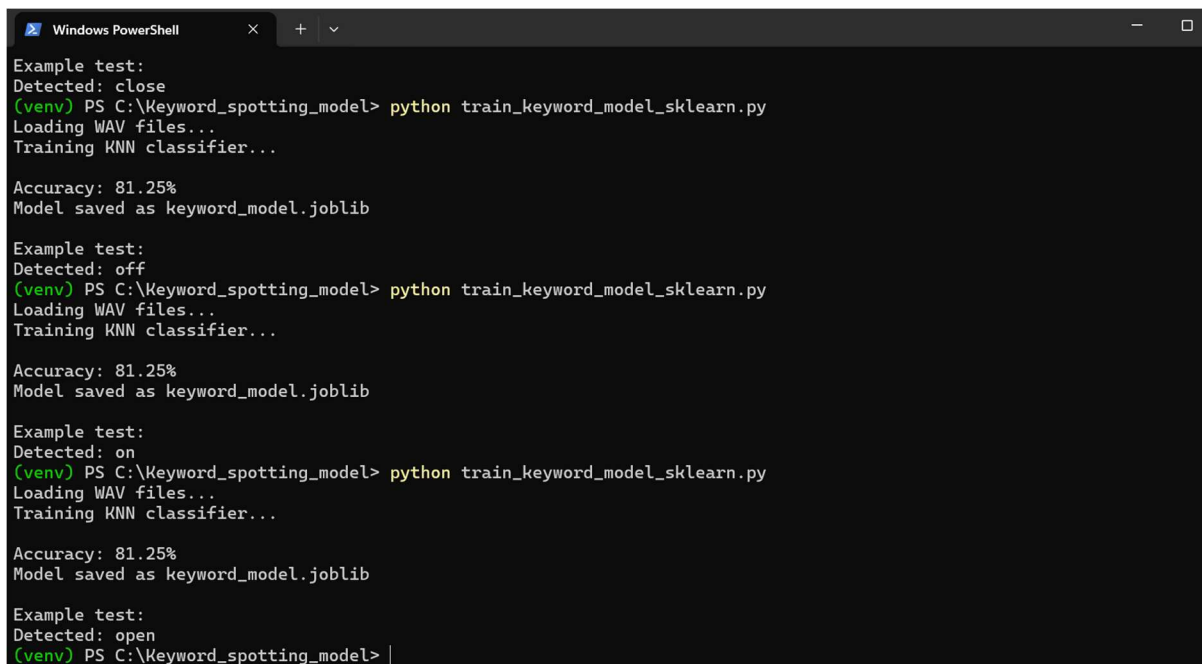
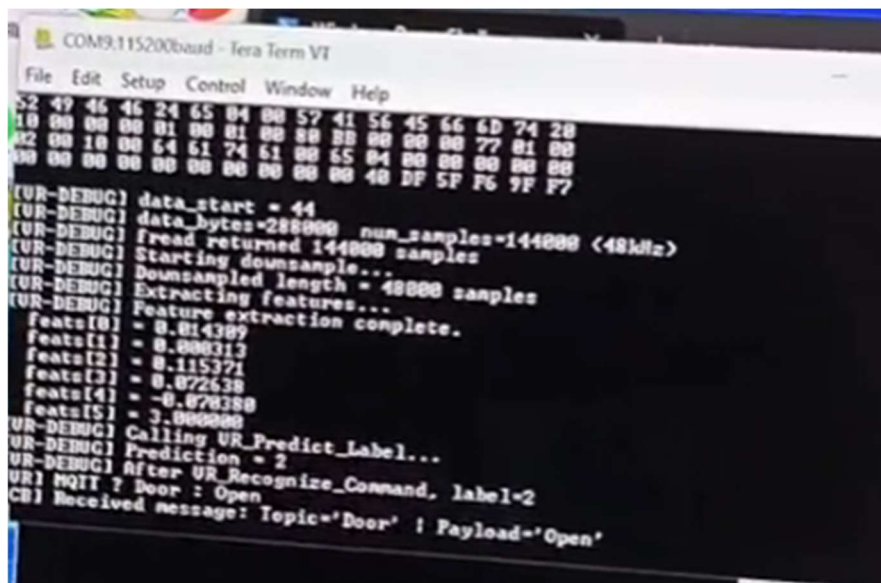


Ilustración 6: Modelo funcional en PC

Aquí tenemos los resultados finales del modelo corriendo en la Pico



```
COM9:115200baud - Tera Term VT
File Edit Setup Control Window Help
52 49 46 46 24 65 04 00 57 41 56 45 66 6D 74 20
10 00 00 00 01 00 01 00 80 80 80 80 80 80 77 01 00
02 00 10 00 64 61 74 61 00 65 04 00 80 80 80 80
00 00 00 00 00 00 00 00 00 00 40 DF 5F F6 7F F7

[UR-DEBUG] data_start = 44
[UR-DEBUG] data_bytes=288000 num_samples=144000 (48kHz)
[UR-DEBUG] fread returned 144000 samples
[UR-DEBUG] Starting downsample...
[UR-DEBUG] Downsampled length = 48000 samples
[UR-DEBUG] Extracting features...
[UR-DEBUG] Feature extraction complete.
feats[0] = 0.014389
feats[1] = 0.000313
feats[2] = 0.115371
feats[3] = 0.072638
feats[4] = -0.070280
feats[5] = 3.000000
[UR-DEBUG] Calling UR_Predict_Label...
[UR-DEBUG] Prediction = 2
[UR-DEBUG] After UR_Recognize_Command, label=2
UR! MQTT ? Door : Open
CB! Received message: Topic='Door' ! Payload='Open'
```

Ilustración 7: Resultados finales en la Pico

Conclusiones

Aprendimos bastante de este proyecto. Principalmente, que integrar un proyecto en dispositivos que no conocemos es extremadamente difícil. Es muy importante conocer completamente el Micro con el que estamos trabajando, o en este caso, la tarjeta de desarrollo. Batallamos bastante con los dispositivos de audio y sus drivers, además de problemas al final al integrar las partes del reconocimiento de voz. Son problemas que se pueden evitar si se trabaja modularmente y se conoce a la perfección el dispositivo y sus periféricos.

Referencias

- OASIS. (2014). *MQTT version 3.1.1*. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- Eclipse Foundation. (s.f.). *Eclipse Mosquitto – An open source MQTT broker*. <https://mosquitto.org>
- OASIS. (2014). *MQTT version 3.1.1. Part 2: Control packet format*. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.pdf>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org>